

agentic-ai – Tata Steel AI Hackathon 2026

Round 2 Submission Documentation

Team / Developer: Likhith Sai Parepalli

Project Name: agentic-ai

Challenge: Agentic AI Challenge – AI-Powered Maintenance Decision Support System

Track: Industrial AI / Predictive Maintenance

Table of Contents

1. [Project Overview](#)
 2. [Problem Statement](#)
 3. [Solution Architecture](#)
 4. [Multi-Agent Workflow](#)
 5. [Technology Stack](#)
 6. [Vision Agent](#)
 7. [RAG Agent](#)
 8. [Anomaly Agent](#)
 9. [Diagnostic Agent](#)
 10. [Risk Scorer](#)
 11. [Report Generator](#)
 12. [Feedback Learning Loop](#)
 13. [Data Flow](#)
 14. [Alerting & Prediction Logic](#)
 15. [Sample Inputs & Outputs](#)
 16. [Installation & Configuration](#)
 17. [Assumptions & Limitations](#)
 18. [Future Scope](#)
-

1. Project Overview

agentic-ai is a **Multimodal Multi-Agent AI Maintenance Decision Support System** designed for steel plant industrial equipment. It acts as an intelligent co-pilot for plant maintenance engineers, enabling

them to diagnose equipment faults, assess risk, predict Remaining Useful Life (RUL), and generate structured maintenance reports – all through a single unified interface.

The system accepts **Voice, Image, Text, CSV sensor data, and PDF documents** as inputs, and delivers:

- Fault identification with root cause analysis
- Risk level classification (LOW / MEDIUM / HIGH / CRITICAL)
- Remaining Useful Life (RUL) prediction from sensor data
- Step-by-step maintenance repair plan
- Spare parts list
- Downloadable structured PDF maintenance report
- Multilingual voice interaction (6 languages: English, Hindi, Odia, Bengali, Dutch, Thai)

Key Differentiators

Feature	agentic-ai	Traditional CMMS
Multimodal Input	✔ Voice + Image + CSV + PDF + Text	✗ Text only
Multi-Agent AI	✔ 8 specialized agents	✗ None
RUL Prediction	✔ XGBoost + Isolation Forest	✗ Manual
RAG Knowledge Base	✔ FAISS + domain SOPs	✗ Static manual
Multilingual	✔ 6 languages via Whisper	✗ English only
Real-time Report	✔ Auto-generated PDF	✗ Manual report
Feedback Learning	✔ Engineer corrections feed back	✗ None

2. Problem Statement

Steel plant equipment – blast furnaces, rolling mills, conveyor systems, hydraulic presses – operate under extreme conditions. Unplanned downtime caused by equipment failure costs steel manufacturers millions per hour. Challenges faced include:

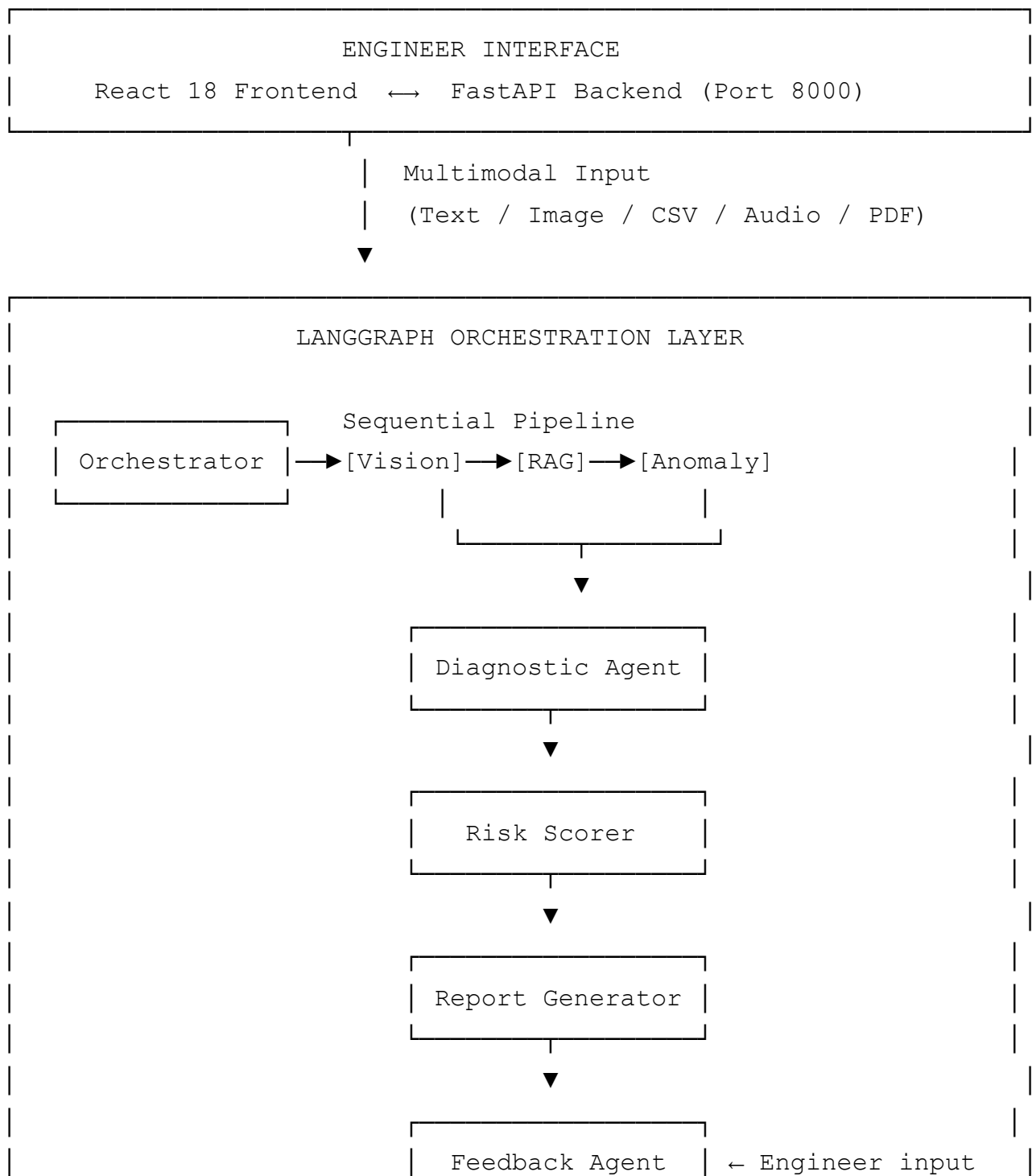
1. **Delayed fault detection** – Engineers visually inspect equipment, missing early-stage failures.
2. **Knowledge silos** – Maintenance knowledge is locked in PDFs, SOPs, and expert minds. Junior engineers lack access.
3. **Reactive maintenance** – Most plants operate on reactive "fix when broken" models, not predictive.
4. **Language barriers** – Plant floor engineers may not communicate in English; diagnosis tools rarely support regional languages.

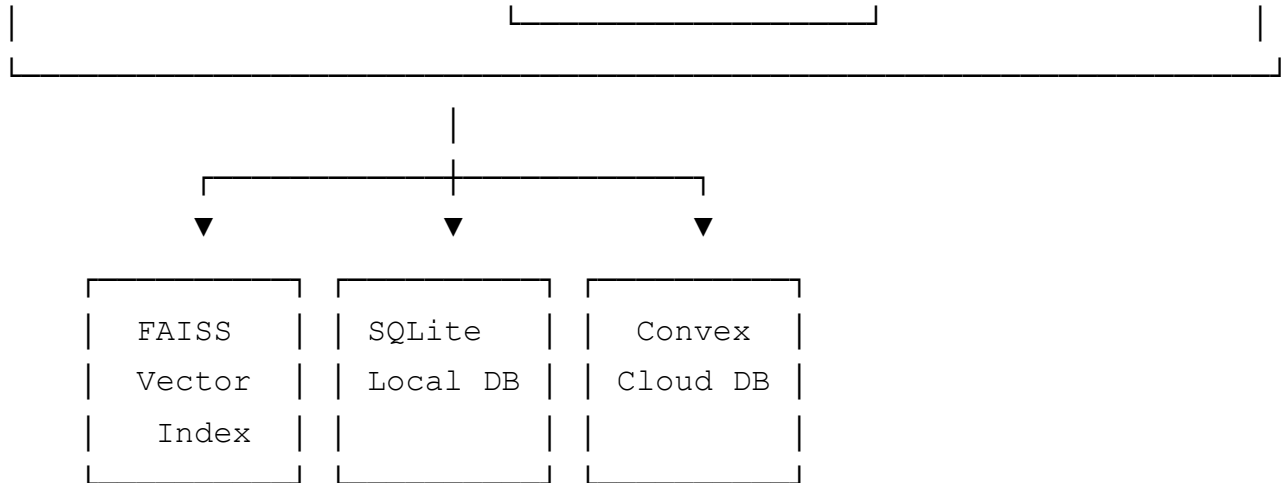
5. **Disconnected data** – Sensor data, images, and maintenance logs exist separately with no unified analysis.

Impact at Tata Steel Scale

- A single blast furnace outage = **₹10–50 crore/day** downtime cost
- Manual MTTD (Mean Time to Detect) = **4–24 hours**
- agentic-ai target MTTD: **< 5 minutes**

3. Solution Architecture





Component Breakdown

Layer	Component	Role
Frontend	React 18 + Vite	Engineer UI, file uploads, result display
API Layer	FastAPI (Python)	REST endpoints, file management, routing
Orchestration	LangGraph 0.2.x	Multi-agent DAG execution
AI Agents	8 specialized agents	See Section 4
LLM	Mistral Small 3.1 (Groq)	Core reasoning
Vision LLM	Gemini 1.5 Flash	Image analysis
STT/TTS	Whisper large-v3 / gTTS	Voice interface
Vector DB	FAISS	Knowledge retrieval
ML Models	Isolation Forest + XGBoost	Anomaly detection + RUL
Database	SQLite + Convex Cloud	Local + cloud persistence

4. Multi-Agent Workflow

The system uses **LangGraph** to define a sequential directed acyclic graph (DAG) of 8 agents that process inputs in order, each agent writing only to its own output key in the shared state.

Agent Execution Order



1. ORCHESTRATOR — Initializes flags (has_image, has_csv, has_docs)
|
| Detects language via Whisper
▼
2. VISION AGENT — Runs only if image uploaded
|
| Uses Gemini 1.5 Flash to analyze equipment photo
|
| Output: visual_findings, damage_areas, severity
▼
3. RAG AGENT — Always runs
|
| Embeds query via sentence-transformers
|
| Searches FAISS index of SOPs, manuals, logs
|
| Output: relevant_docs, maintenance_procedures
▼
4. ANOMALY AGENT — Runs only if CSV uploaded
|
| Isolation Forest anomaly detection
|
| XGBoost RUL prediction
|
| Output: anomaly_score, rul_days, anomalous_sensor
▼
5. DIAGNOSTIC AGENT — Always runs (after Vision+RAG+Anomaly)
|
| Synthesizes ALL agent outputs via Mistral LLM
|
| Output: fault_identified, root_cause, repair_steps
▼
6. RISK SCORER — Calculates composite risk level
|
| Considers: anomaly severity + confidence + RUL
|
| Output: LOW / MEDIUM / HIGH / CRITICAL
▼
7. REPORT GENERATOR — Compiles full structured report
|
| Generates PDF with headers, numbered steps, parts list
|
| Output: report_id, pdf_path, summary
▼
8. FEEDBACK AGENT — Post-repair engineer correction
|
| Updates knowledge base
|
| Logs accuracy metrics

[END]

Shared State Schema (**SteelMindState**)

```
class SteelMindState(TypedDict):  
    # Input  
    query: str  
    language: str                                # hi/or/bn/en/nl/th
```

```
has_image: bool
has_csv: bool
has_docs: bool
image_path: Optional[str]
csv_path: Optional[str]

# Agent Outputs
vision_output: Optional[dict]
rag_context: Optional[list]
anomaly_result: Optional[dict]
diagnosis: Optional[dict]
risk_level: Optional[str] # LOW/MEDIUM/HIGH/CRITICAL
risk_details: Optional[dict]
report: Optional[dict]

# Meta
equipment_id: Optional[str]
equipment_type: Optional[str]
session_id: str
feedback: Optional[dict]
pipeline_errors: List[str]
```

5. Technology Stack

Complete Stack

Category	Technology	Version	Purpose
Agent Framework	LangGraph	0.2.x	Multi-agent DAG orchestration
Core LLM	Mistral Small 3.1 (via Groq)	latest	Diagnosis, reasoning, report generation
Vision LLM	Gemini 1.5 Flash (Google AI)	latest	Equipment image analysis
Speech-to-Text	OpenAI Whisper	large-v3 (local)	Voice input, language detection
Text-to-Speech	gTTS	Python lib	Voice output response
Vector Database	FAISS	1.7.x	Knowledge retrieval (RAG)

Category	Technology	Version	Purpose
Embeddings	sentence-transformers/all-MiniLM-L6-v2	local	Document embedding
Anomaly Detection	Isolation Forest	scikit-learn	Sensor outlier detection
RUL Prediction	XGBoost	latest	Remaining Useful Life regression
Feature Scaling	StandardScaler	scikit-learn	Sensor data normalization
Backend API	FastAPI	0.110.x	REST API layer
Frontend	React 18 + Vite	18 / 8.x	Engineer UI
Styling	Tailwind CSS	latest	UI design system
HTTP Client	Axios	latest	Frontend-backend API calls
Local Database	SQLite	built-in	Session and feedback storage
Cloud Database	Convex	latest	Online sync and history
PDF Generation	fpdf2	latest	Report PDF creation
Container	Docker	latest	Deployment container
PDF Parsing	PyMuPDF (fitz)	latest	Knowledge document ingestion
LangChain	LangChain	0.2.x	LLM chains and vector store
Environment	Python	3.11+	Runtime
Package Manager	pip	latest	Python dependencies
Node Package	npm	latest	Frontend dependencies

API Services Used (All Free Tier)

Service	URL	Used For
Groq API	console.groq.com	Mistral Small 3.1 inference
Google AI Studio	aistudio.google.com	Gemini 1.5 Flash vision
Convex	convex.dev	Cloud database sync

6. Vision Agent

File: `src/agents/vision_agent.py`

LLM: Google Gemini 1.5 Flash

Triggers when: `has_image == True`

Function

Analyzes uploaded equipment photographs to identify visual defects, damage zones, wear patterns, corrosion, leaks, or misalignment.

Input

- Equipment image (JPG/PNG) uploaded by engineer
- Engineer's text query for context

Processing

1. Image is base64-encoded and sent to Gemini 1.5 Flash with a specialized industrial inspection prompt
2. Gemini returns structured JSON: damage areas, severity, recommended inspection zone
3. Output is stored in `state["vision_output"]`

Output Schema

```
{
  "visual_findings": "Significant surface corrosion visible on bearing housing",
  "damage_areas": ["bearing housing", "coupling seal"],
  "visual_severity": "HIGH",
  "recommended_inspection": "Remove bearing housing and inspect inner race for pitting",
  "confidence": 0.87
}
```

System Prompt (from `assets/prompts.md`)

The Vision Agent uses the `VISION_ANALYSIS_PROMPT` which instructs Gemini to:

- Identify specific damaged components
- Estimate severity (NONE/LOW/MEDIUM/HIGH/CRITICAL)

- Suggest inspection zones
 - Output strict JSON for downstream processing
-

7. RAG Agent

File: `src/agents/rag_agent.py`

Embeddings: sentence-transformers/all-MiniLM-L6-v2 (local, offline)

Vector Store: FAISS

Triggers when: Always

Function

Retrieves relevant Standard Operating Procedures (SOPs), maintenance manuals, historical fault records, and domain knowledge to support the Diagnostic Agent.

Knowledge Base Contents (`src/knowledge_base/documents/`)

- Steel plant equipment maintenance SOPs
- Blast furnace operational manuals
- Rolling mill maintenance procedures
- Hydraulic system troubleshooting guides
- Synthetic maintenance logs (historical fault-repair pairs)
- Industry standards (ISO 13374 predictive maintenance)

Processing

1. Engineer's query is embedded via all-MiniLM-L6-v2
2. FAISS performs cosine similarity search over indexed documents
3. Top-k (default: 5) most relevant passages are returned
4. Context is stored in `state["rag_context"]`

Indexing

Run once to index all documents:

```
python src/knowledge_base/index_docs.py
```

Output Schema

```
[
  {
    "source": "blast_furnace_maintenance_sop.pdf",
    "page": 12,
    "content": "For bearing overheating above 85°C, immediately reduce lc
    "similarity_score": 0.94
  }
]
```

8. Anomaly Agent

File: `src/agents/anomaly_agent.py`
Models: Isolation Forest (anomaly) + XGBoost (RUL) + StandardScaler
Triggers when: `has_csv == True`

Function

Detects statistical anomalies in uploaded sensor CSV data and predicts the Remaining Useful Life (RUL) of the equipment.

Supported CSV Formats

The agent auto-resolves column names via aliases – any CSV with numeric sensor data works:

Internal Name	Accepted CSV Column Names
<code>sensor_temperature</code>	temperature, temp, air temperature [k]
<code>sensor_vibration</code>	vibration, vib, tool wear [min]
<code>sensor_pressure</code>	pressure, torque [nm], torque
<code>sensor_rpm</code>	rpm, rotational speed [rpm]
<code>sensor_current</code>	current, power [w], power

ML Models (`src/models/`)

Isolation Forest

- Trained on normal operating sensor readings from AI4I 2020 Predictive Maintenance Dataset
- Detects outlier sensor readings as anomalies

- Decision function: negative score = anomaly, positive = normal
- Normalized to 0.0–1.0 scale (0 = normal, 1 = critical anomaly)

XGBoost RUL Predictor

- Regression model predicting days until failure
- Features: 5 sensor readings (normalized via StandardScaler)
- Output: `rul_days` integer

Output Schema

```
{
  "anomaly_detected": true,
  "anomaly_score": 0.78,
  "severity": "HIGH",
  "anomalous_sensor": "sensor_temperature",
  "current_value": 1580.4,
  "normal_range": "1150-1300°C",
  "rul_days": 4,
  "rul_confidence": 0.85,
  "alert_triggered": true,
  "recommendations": ["Immediate inspection required: RUL critically low."]
}
```

Training the Models

```
python src/models/train_models.py
```

Dataset: Kaggle AI4I 2020 Predictive Maintenance Dataset (10,000 rows, 5 sensor types).

9. Diagnostic Agent

File: `src/agents/diagnostic_agent.py`

LLM: Mistral Small 3.1 (via Groq API)

Triggers when: Always (after Vision + RAG + Anomaly)

Function

The core "brain" of the system. Synthesizes ALL upstream agent outputs into a structured maintenance diagnosis.

Input (from shared state)

- Engineer's query and equipment details
- `vision_output` – visual defects found
- `rag_context` – relevant SOPs and procedures
- `anomaly_result` – sensor anomalies and RUL
- `chat_history` – previous conversation turns (multi-turn support)

Processing

1. Constructs a rich context prompt combining all agent outputs
2. Calls Mistral Small 3.1 via Groq API with `DIAGNOSTIC_PROMPT`
3. Instructs LLM to output strict JSON (fault, root cause, repair steps, parts)
4. Falls back to rule-based diagnosis if LLM fails

Output Schema

```
{
  "fault_identified": "Drive-end bearing failure due to lubrication degra
  "root_cause": "Grease starvation and thermal fatigue from 18 months wit
  "confidence": 0.91,
  "immediate_actions": [
    "Apply LOTO (Lockout/Tagout) immediately",
    "Reduce mill load by 40% if shutdown not possible"
  ],
  "repair_steps": [
    "Step 1: Isolate power and apply LOTO to rolling mill",
    "Step 2: Remove coupling and housing to access bearing",
    "Step 3: Extract bearing using bearing puller SKF TMMP 20",
    "Step 4: Clean shaft and housing, measure fits",
    "Step 5: Install new bearing SKF 6318/C3 with correct torque"
  ],
  "spare_parts_needed": [
    { "name": "SKF Deep Groove Bearing 6318/C3", "part_number": "SKF-6318-
    { "name": "High-Temperature Bearing Grease", "part_number": "GREASE-HT
  ],
  "estimated_repair_time": "4-6 hours",
  "long_term_recommendations": "Implement quarterly bearing greasing sche
```

```
"sources_cited": ["bearing_maintenance_sop.pdf p.12", "rolling_mill_mar
}
```

10. Risk Scorer

File: `src/agents/risk_scorer.py`

Triggers when: After Diagnostic Agent

Function

Calculates a composite risk level based on diagnostic confidence, anomaly severity, and RUL prediction.

Risk Matrix

Condition	Risk Level
RUL < 3 days OR anomaly_score > 0.85	 CRITICAL
RUL < 7 days OR anomaly_score > 0.70	 HIGH
RUL < 14 days OR anomaly_score > 0.60	 MEDIUM
All other conditions	 LOW

Additional Escalation Rules

- `force_critical = True` (set by Anomaly Agent when RUL < 7) always overrides to CRITICAL
- `escalate_to_supervisor` flag set when CRITICAL
- `urgency_hours` calculated: CRITICAL=2h, HIGH=8h, MEDIUM=48h, LOW=168h

Output Schema

```
{
  "risk_level": "HIGH",
  "risk_score": 0.78,
  "urgency_hours": 8,
  "escalate_to_supervisor": false,
  "risk_rationale": "Anomaly score 0.78 exceeds HIGH threshold; RUL of 4
}
```

11. Report Generator

File: `src/agents/report_generator.py`

Library: fpdf2

Triggers when: After Risk Scorer

Function

Compiles all agent outputs into a structured, professionally formatted PDF maintenance report.

PDF Structure

1. **Blue title header bar** – "agentic-ai | Maintenance Diagnostic Report"
2. **Metadata row** – Report ID, timestamp, Tata Steel attribution
3. **Equipment & Risk Summary** – Equipment ID, type, risk level, urgency
4. **Executive Summary** – One-paragraph fault description
5. **Fault Diagnosis** – Fault, root cause, confidence, repair time
6. **Immediate Actions** – Numbered list
7. **Step-by-Step Repair Plan** – Numbered steps
8. **Safe Component Change Plan** – LOTO procedure steps
9. **Spare Parts Required** – Bullet list with part numbers
10. **Sensor Anomaly & RUL** – Score, RUL, anomalous sensor
11. **Long-Term Recommendations**
12. **Sources Cited**
13. **Footer** – Italic attribution line

Download

GET /report/{session_id}

→ SteelMind-Maintenance-Report-RPT-YYYYMMDD-HHMMSS.pdf

12. Feedback Learning Loop

File: `src/agents/feedback_agent.py`

Triggers when: Engineer submits post-repair feedback

Function

After a repair is completed, the engineer submits feedback on whether the AI diagnosis was correct, what actually happened, and the repair outcome. This feedback:

1. Logs accuracy metrics to SQLite
2. Syncs to Convex cloud database
3. Updates the FAISS knowledge base with correct fault-repair pairs
4. Enables continuous improvement without retraining

Feedback API

POST /feedback

```
report_id: str
diagnosis_correct: bool
actual_fault: str (if incorrect)
outcome: RESOLVED | ESCALATED | MONITORING
downtime_hours: float
engineer_notes: str
```

UI Feedback Buttons

Three buttons appear after diagnosis:

-  **Resolved** – Repair succeeded, diagnosis was accurate
 -  **Escalated** – Required senior engineer involvement
 -  **Monitoring** – Issue persists, still observing
-

13. Data Flow

Complete Request Lifecycle

[1] Engineer Input (UI)

- └─ Text query typed
- └─ Equipment image uploaded
- └─ Sensor CSV uploaded
- └─ Equipment ID entered

[2] Frontend → Backend

```
POST /diagnose (multipart/form-data)
└─ query: str
└─ equipment_id: str
```

- └─ image: File (JPG/PNG)
- └─ csv_file: File (CSV)
- └─ chat_history: JSON

[3] File Processing (main.py)

- └─ Image saved to uploads/images/{uuid}.jpg
- └─ CSV saved to uploads/csv/{uuid}.csv
- └─ Session ID generated (hex[:12])

[4] LangGraph Pipeline Execution

- └─ execute_steelmind_pipeline(initial_state)
 - └─ Orchestrator → sets has_image, has_csv, has_docs
 - └─ Vision Agent → Gemini 1.5 Flash → vision_output
 - └─ RAG Agent → FAISS search → rag_context
 - └─ Anomaly Agent → Isolation Forest + XGBoost → anomaly_result
 - └─ Diagnostic Agent → Mistral LLM → diagnosis
 - └─ Risk Scorer → rule-based → risk_level
 - └─ Report Generator → fpdf2 → PDF file

[5] Response to Frontend

```
{  
    session_id, status, diagnosis,  
    risk_level, anomaly_result, report  
}
```

[6] Frontend Display

- └─ Conversation log updated
- └─ Diagnosis & Action Plan card rendered
- └─ Risk badge shown (color-coded)
- └─ Sensor Anomaly & RUL section shown
- └─ Download PDF button enabled

[7] Persistence

- └─ SQLite → local session history
- └─ Convex → cloud sync (history, feedback)

14. Alerting & Prediction Logic

Anomaly Detection (Isolation Forest)

The Isolation Forest model was trained on 10,000 sensor readings from the AI4I 2020 Predictive Maintenance Dataset. It detects outliers by randomly partitioning features – anomalous readings are isolated in fewer splits.

Scoring:

```
raw_score = decision_function(sensor_readings)
# negative = anomaly, positive = normal

anomaly_score = max(0.0, min(1.0, 0.5 - (raw_score × 2.0)))
# 0.0 = perfectly normal, 1.0 = extreme anomaly
```

RUL Prediction (XGBoost)

Features: [temperature, vibration, pressure, rpm, current] (normalized)
Target: days_until_failure
Model: XGBoost Regressor (trained on synthetic RUL dataset)
Output: rul_days (int, capped at 0)

Alert Thresholds

Threshold	Condition	Action
<code>anomaly_score > 0.85</code>	Extreme anomaly	Force CRITICAL risk
<code>rul_days < 7</code>	Imminent failure	<code>alert_triggered = True</code> , Force CRITICAL
<code>rul_days < 3</code>	Failure today/tomorrow	Immediate shutdown recommended
<code>risk_level == CRITICAL</code>	Combined criteria	<code>escalate_to_supervisor = True</code>

Sensor Deviation Detection

The most anomalous sensor is identified by:

```
deviation[sensor] = |latest_value - historical_mean| / historical_std
anomalous_sensor = argmax(deviation)
```

Multi-turn Conversation

The system maintains `chat_history` across sessions. Each new query is sent with previous context, enabling the Diagnostic Agent to provide follow-up reasoning without losing context.

15. Sample Inputs & Outputs

Sample Input 1 – Text + Image Query

Input:

Equipment ID: RM-204

Query: "Rolling mill making grinding noise and vibration at drive end bearing.

Temperature shows 78°C – 15 degrees above normal. Diagnose."

Image: bearing_photo.jpg (shows surface rust and grease leakage)

Output:

Risk Level: HIGH 

Fault: Drive-end bearing failure – grease starvation + thermal fatigue

Root Cause: Bearing seal degraded; water ingress caused grease washout

Confidence: 91%

Immediate Actions:

1. Apply LOTO to Rolling Mill RM-204
2. Reduce load by 40% if immediate shutdown impossible

Repair Steps:

1. Isolate power, apply LOTO
2. Remove coupling and bearing housing
3. Extract bearing (SKF TMMP 20 puller)
4. Clean shaft, measure fits (should be k6)
5. Install new SKF 6318/C3 bearing with 280Nm torque
6. Apply high-temperature grease (80% fill)
7. Reinstall housing, run for 2 hours at reduced load
8. Verify temperature stabilizes below 65°C

Spare Parts:

- 1× SKF Deep Groove Bearing 6318/C3
- 2× High-Temperature Bearing Grease 250g
- 1× Bearing Housing Seal Kit

Estimated Repair Time: 4-6 hours

Sample Input 2 – CSV Sensor Data

Input CSV (sensor_data.csv):

```
timestamp,sensor_temperature,sensor_vibration,sensor_pressure,sensor_rpm,
2026-06-14 08:00,1258,1.2,175,1490,88
2026-06-14 09:00,1289,1.8,172,1485,91
2026-06-14 10:00,1580,3.2,71,0,628
2026-06-14 11:00,1592,3.8,68,0,641
```

Output:

Anomaly Score: 0.82 (HIGH)

RUL: 4 days

Anomalous Sensor: sensor_temperature (current: 1592°C, normal: 1150-1300°C)

Alert:  RUL < 7 days – Immediate inspection required

Sample PDF Report

Downloaded as: [SteelMind-Maintenance-Report-RPT-20260614-213325.pdf](#)

Sections: Title Header | Equipment Summary | Executive Summary | Fault Diagnosis | Immediate Actions | Repair Steps | Spare Parts | RUL | Sources | Footer

16. Installation & Configuration

Prerequisites

Python 3.11+

Node.js 18+

npm

Git

Step 1: Clone Repository

```
git clone https://github.com/likhith816/agent-ai
cd agent-ai
```

Step 2: Create Virtual Environment

```
python -m venv venv
# Windows
venv\Scripts\activate
# Linux/Mac
source venv/bin/activate
```

Step 3: Install Python Dependencies

```
pip install -r requirements.txt
```

Step 4: Configure API Keys

Copy the example `.env` file:

```
cp .env.example .env
```

Edit `.env` with your API keys:

```
GROQ_API_KEY=your_groq_api_key          # https://console.groq.com (free)
GOOGLE_API_KEY=your_google_api_key      # https://aistudio.google.com (fr
MISTRAL_API_KEY=your_mistral_api_key    # https://console.mistral.ai (fre
```

Step 5: Train ML Models

```
python src/models/train_models.py
```

Expected output:

- ✓ Isolation Forest trained and saved to `src/models/isolation_forest.pkl`
- ✓ XGBoost RUL model trained and saved to `src/models/rul_model.pkl`
- ✓ StandardScaler saved to `src/models/scaler.pkl`

Step 6: Index Knowledge Base

```
python src/knowledge_base/index_docs.py
```

Expected output:

✅ Indexed 47 documents → src/knowledge_base/faiss_index/

Step 7: Start Backend

```
# Option A: Direct
python main.py

# Option B: Uvicorn (production-like)
uvicorn main:app --reload --port 8000
```

Backend live at: <http://localhost:8000>

API documentation: <http://localhost:8000/docs>

Step 8: Start Frontend

```
cd frontend
npm install
npm run dev
```

Frontend live at: <http://localhost:5173>

Step 9: Quick Start (Windows)

Double-click `START_SERVERS.bat` to launch both servers in independent windows.

Docker Deployment (Optional)

```
docker build -t agentic-ai .
docker run -p 8000:8000 --env-file .env agentic-ai
```

API Endpoints

Method	Endpoint	Description
POST	/diagnose	Main diagnosis (text + files)
POST	/voice	Voice audio diagnosis
POST	/feedback	Submit engineer feedback
GET	/report/{id}	Download PDF report
GET	/health	Health check
GET	/history	Session history
GET	/equipment	Equipment list
GET	/predictions	RUL predictions
GET	/summary	Plant health KPIs
POST	/guide	Onboarding chatbot

17. Assumptions & Limitations

Assumptions

- Sensor column names** – The system auto-maps common column name variants, but CSV data must contain at least 5 numeric sensor columns.
- Minimum CSV rows** – Anomaly detection requires at least 10 rows of sensor data for meaningful predictions.
- Language detection** – Whisper auto-detects language from audio; text queries are assumed English unless voice input is used.
- Knowledge base** – RAG retrieval quality depends on the quality and coverage of indexed documents. Thin knowledge bases will produce less precise recommendations.
- ML model scope** – The Isolation Forest and XGBoost models were trained on the AI4I 2020 Predictive Maintenance Dataset (milling machine context). Performance on other equipment types may vary until retrained on plant-specific data.
- Network connectivity** – Gemini and Groq API calls require internet connectivity. The system degrades gracefully with rule-based fallbacks if APIs are unavailable.
- Image quality** – Vision Agent performance degrades with low-resolution or obscured images. Minimum recommended resolution: 640×480px.

Current Limitations

Limitation	Mitigation
------------	------------

Limitation	Mitigation
No real-time sensor streaming	Accepts CSV uploads; streaming can be added via WebSocket
ML models not plant-specific	Retrain with actual Tata Steel sensor data for production use
No SCADA/DCS integration	REST API allows integration; SCADA adapters can be built
Voice only via file upload	Real-time mic recording can be added with MediaRecorder API
Knowledge base is synthetic	Replace with actual Tata Steel SOPs for production
Single-user SQLite	Scale to PostgreSQL/Convex for multi-user production
Whisper runs locally	Can be replaced with cloud STT for lower memory footprint

18. Future Scope

Short Term (1–3 months)

1. **Real-time sensor streaming** – WebSocket endpoint for live sensor feed from SCADA/DCS systems instead of CSV upload
2. **SCADA/DCS integration** – OPC-UA or MQTT adapter to pull sensor data directly from plant floor
3. **Plant-specific ML retraining** – Retrain Isolation Forest and XGBoost on actual Tata Steel sensor history
4. **Mobile app** – React Native app for field engineers to submit queries and receive diagnoses on mobile devices

Medium Term (3–6 months)

5. **Digital Twin integration** – Connect to 3D digital twin models of equipment for visual fault overlay
6. **Predictive maintenance scheduling** – Auto-generate maintenance work orders in SAP PM / IBM Maximo based on RUL predictions
7. **Fleet monitoring dashboard** – Monitor all plant equipment simultaneously with real-time RUL countdowns
8. **Spare parts auto-ordering** – Integrate with procurement API to auto-raise purchase orders for spare parts identified in the diagnosis
9. **Multi-plant deployment** – Extend to multiple Tata Steel plants with centralized knowledge sharing

Long Term (6–12 months)

10. **Self-improving knowledge base** – Active learning loop where engineer corrections automatically refine the RAG index and retrain models

Contact & Attribution

Developer: Likhith Sai Parepalli

GitHub: <https://github.com/likhith816>

Project: agentic-ai

Event: Tata Steel AI Hackathon 2026 — Round 2: Agentic AI Challenge

Built with LangGraph · Mistral · Gemini · FAISS · XGBoost · FastAPI · React